MODULE *AddFailStopCR*

EXTENDS *CR*, *FiniteSets*

VARIABLE
STEPS, A counter that increases by 1 on each step
STEP_TAKEN, A string variable that stores the current step being taken
Current_Tail, stores current position of *Tail* across failures and normal *operations.InCR*, *TAILisaconstantthatcannotchange*.
REMOVED_INDICES, Stores set of indices in *srvVals* that are *failed/deletedinFailureAndRecoveryStep*
srvHist, Maintains sequence of write updates for each server
srvSent, Maintains sequence of write updates sent by a server for which *ithasnotreceivedanAck*
srvAck, Stores set of updates for which a server has received an *Ack*, *basedonwhichcorrespondingentrycanbedeletedfromsrvSent*
srvMaxWriteSeqNumReceived, Max sequence number of the last update *receivedbyaserver*
False_Tail, Tracks last element in a chain which has not caught up. If *allservershavecaughtup*, *thisisequaltoCurrent_Tail*
NUM_SERVERS_DELETED, Var to keep track of how many servers have been *deletedsincethebeginning*
NUM_SERVERS_ADDED Var to keep track of how many servers have been added *sincethebeginning*

CONSTANT *DEBUG_ENABLE*

Helper function to create a set from elements of a Sequence
SequencetoSet(*seq*) $\triangleq$
$\{seq[i] : i \in \text{DOMAIN } seq\}$

Sync reads from *resps* to last server, needed in case *Tail* fails
SyncSrvRepliedReadsFromResps(*srvRepliedState*, *respsState*) $\triangleq$
$[\ i \in 1 \,..\, Len(srvRepliedState)$
$\mapsto$
IF
$i = Len(srvRepliedState)$
THEN
$[\ r \mapsto$
$srvRepliedState[i].r$
$\cup$
$\{$
$r.id : r \in$
$\{\ resp \in \{respsState[j] : j \in \text{DOMAIN } respsState\}$
:
$resp \in RdRes$

```
                        }
                    }
                ,
                w ↦ srvRepliedState[i].w]
        ELSE
            srvRepliedState[i]]
```

Helper function to return a new sequence from elements in argument 1 *seq*, that are not present in argument 2 *seq*

```
RemoveElementsfromSequence(seq_to_remove_from, filter_seq) ≜
    [i ∈ 1 .. Cardinality({j ∈ DOMAIN seq_to_remove_from :
seq_to_remove_from[j] ∉  SequencetoSet(filter_seq)})
        ↦
            LET
                validindexes ≜
                    [j ∈ 1 .. Len(seq_to_remove_from)
                        ↦
                            IF seq_to_remove_from[j] ∈
SequencetoSet(filter_seq) THEN 0 ELSE  j]

                  filteredindexes ≜
                      SelectSeq(validindexes, LAMBDA x : x > 0)

            IN

                seq_to_remove_from[filteredindexes[i]]
    ]
```

After Failure of a Server S between $S-$ *and* $S+$, there can be updates that S- sent to S, but were not received by $S+$. As part of Failure Recovery, the updates not seen by $S+$, but sent by $S-$ are sent again to $S+$, to make the *Inprocess* Requests *Invariant* True. The suffix of *srvSent* of $S-$ *to* be sent is calculated based on *srvMaxWriteSeqNumReceived* stored by $S-$, and subtracting it from *srvHist* of $S-$, which is the maximum writes sent by $S-$.

```
FailureRecovery(server_state, server_history, server_sent,
server_max_write_seq_num, temp_tail) ≜
    [
        i ∈ 1 .. Len(server_state)
            ↦
                IF (i = 1 ∨ i > temp_tail)
                THEN
                    server_state[i]
                ELSE
                    server_state[i] ∘

                        LET
```

$CurrentSequenceCount \triangleq Len(server_history[i] \circ RemoveElementsfromSequence(server_state[i], server_history[i]))$

$CurrentSequenceCount \triangleq server_max_write_seq_num[i]$

$PreviousSequenceCount \triangleq Len(server_history[i-1])$

$PreviousSequenceCount \triangleq server_max_write_seq_num[i-1]$

$LengthofPrevSentToTransfer \triangleq PreviousSequenceCount - CurrentSequenceCount$

IN

$SubSeq(server_sent[i-1],\ Len(server_sent[i-1]) - LengthofPrevSentToTransfer + 1,\ LengthofPrevSentToTransfer\ )$

]

State of multiple servers and stored in sequence. Failing a server means deleting an element from that sequence, so that successor and predecessor relationships are implicit based on adjacent neighbors in the new sequence. For a failure, from each srv variable, elements are deleted that are passed in set *failed_servers*.

$FailServer(server_state,\ failed_servers) \triangleq$
$[i \in 1\ ..\ (Len(server_state) - Cardinality(failed_servers))$
$\mapsto$
LET
$validindexes \triangleq$
$[j \in 1\ ..\ Len(server_state)$
$\mapsto$
IF $j \in failed_servers$ THEN 0 ELSE $j]$
$filteredindexes \triangleq$
$SelectSeq(validindexes,$ LAMBDA $x : x > 0)$
IN
$server_state[filteredindexes[i]]]$

Safety Property added separately as invariant in cfg file

$InprocessRequestsSafety \triangleq$
$\forall\, i \in (1\ ..\ Current_Tail) :$
$\forall\, j \in (i\ ..\ Current_Tail) :$
$srvHist[i] = srvHist[j] \circ RemoveElementsfromSequence(srvSent[i], srvHist[j])$

Safety Property added separately as invariant in cfg file

$UpdatePropagationSafety \triangleq$
$\forall\, i \in (1\,..\,Current_Tail) :$
$\forall\, j \in (i\,..\,Current_Tail) :$
$SubSeq(srvHist[i],\, 1,\, Len(srvHist[j])) = srvHist[j]$

Debug *Invariant* only used when *DEBUG_ENABLE* constant is 1 in cfg. Does not affect actual *FailStopCR* spec

$DebugTypeOKInvariants \triangleq$
IF $DEBUG_ENABLE = 1$
THEN
$\wedge\, STEPS \geq 0$
$\wedge\, \neg(STEPS = 12)$
$\wedge\, \neg($
$\wedge\, False_Tail > Current_Tail \wedge STEP_TAKEN =$
"FailureAndRecoveryStep"
)

ELSE
TRUE

Safety properties for all Variables

$AddFailStopCRTypeOK \triangleq$
$\wedge\, srvVals \in [(1\,..\,False_Tail) \to VALS]$
$\wedge\, srvReqs \in [(1\,..\,False_Tail) \to Seq(Req)]$
$\wedge\, srvReplied \in [(1\,..\,False_Tail) \to [r : \text{SUBSET } RID,\, w : \text{SUBSET } WID]]$
$\wedge\, resps \in Seq(Res)$
$\wedge\, reqs \subseteq Req$
$\wedge\, srvHist \in [(1\,..\,False_Tail) \to Seq(WrReq)]$
$\wedge\, srvMaxWriteSeqNumReceived \in [(1\,..\,False_Tail) \to Nat]$
$\wedge\, srvSent \in [(1\,..\,False_Tail) \to Seq(WrReq)]$
$\wedge\, srvAck \in [(1\,..\,False_Tail) \to \text{SUBSET } WID]$
$\wedge\, REMOVED_INDICES \in \text{SUBSET } (Nat \setminus \{0\})$
$\wedge\, STEPS \in Nat$

Setting initial values of all variables

$AddFailStopCRInit \triangleq$
$\wedge\, CRInit$
$\wedge\, Current_Tail = TAIL$
$\wedge\, srvHist = [s \in (1\,..\,Current_Tail) \mapsto \langle\rangle]$
$\wedge\, srvSent = [s \in (1\,..\,Current_Tail) \mapsto \langle\rangle]$
$\wedge\, srvAck = [s \in (1\,..\,Current_Tail) \mapsto \{\}]$
$\wedge\, srvMaxWriteSeqNumReceived = [s \in (1\,..\,Current_Tail) \mapsto 0]$
$\wedge\, STEPS = 0$
$\wedge\, REMOVED_INDICES = \{\}$
$\wedge\, STEP_TAKEN =$ "INIT"

$\land$ $False_Tail \quad = TAIL$
$\land$ $NUM_SERVERS_DELETED = 0$
$\land$ $NUM_SERVERS_ADDED = 0$

Failure and Recovery of new successor/predecessor relationships are done in this step

$FailureAndRecoveryStep \triangleq$
$\land$ UNCHANGED $\langle reqs, resps, NUM_SERVERS_ADDED\rangle$
$\land$ $STEP_TAKEN' =$ "FailureAndRecoveryStep"
$\land \exists\, indices_to_remove \in$ SUBSET $(1 \,..\, Current_Tail)$:
$\land$ $Cardinality(indices_to_remove) < Current_Tail$
$\land$ $Cardinality(indices_to_remove) > 0$
$\land$ $REMOVED_INDICES' = indices_to_remove$
$\land$ $NUM_SERVERS_DELETED' = NUM_SERVERS_DELETED +$ $Cardinality(indices_to_remove)$
$\land$ $srvVals' = FailServer(srvVals, indices_to_remove)$
$\land$ $srvHist' = FailServer(srvHist, indices_to_remove)$
$\land$ $srvSent' = FailServer(srvSent, indices_to_remove)$
$\land$ $srvAck' = FailServer(srvAck, indices_to_remove)$
$\land$ $Current_Tail' = Current_Tail - Cardinality(indices_to_remove)$
$\land$ $False_Tail' = False_Tail - Cardinality(indices_to_remove)$
$\land$ $srvMaxWriteSeqNumReceived' = FailServer(srvMaxWriteSeqNumReceived,$ $indices_to_remove)$
$\land$ $srvReqs' = FailureRecovery(FailServer(srvReqs, indices_to_remove),$ $FailServer(srvHist, indices_to_remove), \ FailServer(srvSent,$ $indices_to_remove), FailServer(srvMaxWriteSeqNumReceived, indices_to_remove),$ $Current_Tail - Cardinality(indices_to_remove))$
$\land$ $resps' = resps \circ RemoveElementsfromSequence(FailServer(srvSent,$ $indices_to_remove)[Current_Tail'], resps)$ Sending Writes to *resps* if *Tail* is *removed*
$\land$ $srvReplied' = FailServer(srvReplied, indices_to_remove)$
$\land$ $srvReplied' = SyncSrvRepliedReadsFromResps(FailServer(srvReplied,$ $indices_to_remove), resps)$ * Updating *srvReplied* reads in the new pre-tail server from *resps*(*Needed for refinement mapping with SS*)

$AddNewServertoChain(num_new_servers) \triangleq$
$\land$ $STEP_TAKEN' =$ "AddNewServertoChain"
$\land$ $NUM_SERVERS_ADDED' = NUM_SERVERS_ADDED + num_new_servers$
$\land$ $srvVals' = [i \in 1 \,..\, (Len(srvVals) + num_new_servers) \mapsto$ IF $i \leq$ $Len(srvVals)$ THEN $srvVals[i]$ ELSE $INITVAL]$
$\land$ $srvReqs' = [i \in 1 \,..\, (Len(srvReqs) + num_new_servers) \mapsto$ IF $i \leq$ $Len(srvReqs)$ THEN $srvReqs[i]$ ELSE $\langle\rangle]$
$\land$ $srvReplied' = [i \in 1 \,..\, (Len(srvReplied) + num_new_servers) \mapsto$ IF $i \leq$ $Len(srvReplied)$ THEN $srvReplied[i]$ ELSE $[r \mapsto \{\}, w \mapsto \{\}]]$
$\land$ $srvHist' = [i \in 1 \,..\, (Len(srvHist) + num_new_servers) \mapsto$ IF $i \leq$

$Len(srvHist)$ THEN $srvHist[i]$ ELSE $\langle\rangle]$
$\land\ srvSent' = [i \in 1\,..\,(Len(srvSent) + num_new_servers) \mapsto$ IF $i \leq Len(srvSent)$ THEN $srvSent[i]$ ELSE $\langle\rangle]$
$\land\ srvAck' = [i \in 1\,..\,(Len(srvAck) + num_new_servers) \mapsto$ IF $i \leq Len(srvAck)$ THEN $srvAck[i]$ ELSE $\{\}]$
$\land\ srvMaxWriteSeqNumReceived' = [i \in 1\,..\,(Len(srvMaxWriteSeqNumReceived) + num_new_servers) \mapsto$ IF $i \leq Len(srvMaxWriteSeqNumReceived)$ THEN $srvMaxWriteSeqNumReceived[i]$ ELSE $0]$
$\land$ UNCHANGED $\langle reqs, resps, REMOVED_INDICES, Current_Tail, NUM_SERVERS_DELETED\rangle$
$\land\ False_Tail' = False_Tail + num_new_servers$

$ShiftCurrentTailToNewServer \triangleq$
$\land\ STEP_TAKEN' =$ "ShiftCurrentTailToNewServer"
$\land\ Current_Tail < Len(srvVals)$
$\land\ srvHist[Current_Tail] = srvHist[Current_Tail + 1] \circ RemoveElementsfromSequence(srvSent[Current_Tail], srvHist[Current_Tail + 1])$ $CheckingInProcessRequestsInvariantwhichimpliesTailcanbeshiftedtonew node.$

Applying elements of Sent from old *Tail* to new *Tail*, such that both *srvHist* of new and old *Tail* are same

$\land\ srvHist' = [srvHist$ EXCEPT $![Current_Tail + 1] = srvHist[Current_Tail + 1] \circ RemoveElementsfromSequence(srvSent[Current_Tail], srvHist[Current_Tail + 1])$]
$\land\ srvSent' = srvSent$

$\land\ srvReqs' = srvReqs$
$\land\ srvReqs' = [srvReqs$ EXCEPT $![Current_Tail + 1] = srvSent[Current_Tail]$]

$\land\ srvReplied' =$
[
$s \in 1\,..\,False_Tail$
$\mapsto$
IF $s = Current_Tail$
THEN
[
$r \mapsto \{\}$, Deleting read history from server which is not $goingtobeTAILinthenextstate$
$w \mapsto srvReplied[s].w$
]
ELSE
IF $s = Current_Tail + 1$ New *Tail* in next state
THEN
[

```
                    Syncing read history from old Tail to new Tail, can
                    also be deleted, as history can be large, and does
                    not affect data consistency of SS
                  r ↦ srvReplied[s − 1].r,

                    New Tail aready has partial write history from old
                    Tail sue to srvHist sync. The rest will be updated,
                    when writes received from srvSent of old Tail will be
                    applied
                  w ↦ srvReplied[s].w ∪ srvReplied[s − 1].w
              ]

          ELSE
            srvReplied[s]
      ]
  ∧ Current_Tail′ = Current_Tail + 1
  ∧ srvVals′ =
      [
        srvVals EXCEPT ![Current_Tail + 1] =
        IF Len(srvSent[Current_Tail]) > 0
         THEN
           srvSent[Current_Tail][Len(srvSent[Current_Tail])].v
         ELSE
        srvVals[Current_Tail + 1]
      ]

  ∧ UNCHANGED ⟨reqs, resps, REMOVED_INDICES, False_Tail, srvAck,
srvMaxWriteSeqNumReceived, NUM_SERVERS_ADDED, NUM_SERVERS_DELETED⟩

SyncHistoryWithNewServer ≜
  ∧ STEP_TAKEN′ = "SyncHistoryWithNewServer"
  ∧ Current_Tail < False_Tail
  ∧ srvHist[Current_Tail] ≠ srvHist[Current_Tail + 1] ∘
RemoveElementsfromSequence(srvSent[Current_Tail], srvHist[Current_Tail + 1])
CheckingInprocessrequestinvariantistillnotTRUE, keepsyncingHistory
fromTailtonewserver.
  ∧ srvHist′ =
      [
        s ∈ 1 .. False_Tail
        ↦
        IF
            ∧ s > Current_Tail
            ∧ Len(srvHist[s]) < Len(srvHist[s − 1])
         THEN
```

```
            Append(srvHist[s], srvHist[s − 1][Len(srvHist[s]) + 1])
        ELSE
            srvHist[s]
    ]
∧  srvMaxWriteSeqNumReceived′ =
        [
            s    ∈ 1 .. False_Tail
            ↦
            IF
                 ∧ s > Current_Tail
                 ∧ Len(srvHist[s]) < Len(srvHist[s − 1])
             THEN
                srvMaxWriteSeqNumReceived[s] + 1
             ELSE
                srvMaxWriteSeqNumReceived[s]
        ]
∧  srvReplied′ =
    [
        s    ∈ 1 .. False_Tail
        ↦
        IF
            s > Current_Tail ∧ Len(srvHist[s]) < Len(srvHist[s − 1])
         THEN
            [
                r  ↦ srvReplied[s].r,
                w ↦
                    srvReplied[s].w
                    ∪
                    {srvHist[s − 1][Len(srvHist[s]) + 1].id}]
         ELSE
            srvReplied[s]
    ]
∧  srvVals′ =
        [
            s    ∈ 1 .. False_Tail
            ↦
            IF
                 ∧ s > Current_Tail
                 ∧ Len(srvHist[s]) < Len(srvHist[s − 1])
             THEN
                srvHist[s − 1][Len(srvHist[s]) + 1].v
             ELSE
                srvVals[s]
```

]

$\land$ UNCHANGED $\langle Current_Tail, False_Tail, REMOVED_INDICES, reqs, resps, srvAck, srvReqs, srvSent, NUM_SERVERS_ADDED, NUM_SERVERS_DELETED\rangle$

Modifying *IssueRead* step from *CR* as it used a constant *TAIL*, but due to failures, *Current_Tail* is used here which is updated on each failure

$ModifiedCRIssueRead(rid) \triangleq$
LET $op \triangleq [id \mapsto rid]$IN
$\land\ op \notin reqs$
$\land\ reqs' = reqs \cup \{op\}$
$\land\ srvReqs' = [srvReqs$ EXCEPT $![Current_Tail] = Append(@, op)]$
$\land$ UNCHANGED $\langle srvVals, srvReplied, resps, NUM_SERVERS_ADDED, NUM_SERVERS_DELETED\rangle$

Modifying *IssueWrite* step from *CR* to increase *srvMaxWriteSeqNumReceived* on *HEAD*

$ModifiedCRIssueWrite(wid) \triangleq$
$\exists\, v \in VALS :$
LET $op \triangleq [id \mapsto wid, v \mapsto v]$IN
$\land\ op \notin reqs$
$\land\ reqs' = reqs \cup \{op\}$
$\land\ srvReqs' = [srvReqs$ EXCEPT $![HEAD] = Append(@, op)]$
$\land\ srvMaxWriteSeqNumReceived' = [srvMaxWriteSeqNumReceived$ EXCEPT $![HEAD] = srvMaxWriteSeqNumReceived[HEAD] + 1]$
$\land$ UNCHANGED $\langle srvVals, srvReplied, resps, NUM_SERVERS_ADDED, NUM_SERVERS_DELETED\rangle$

Modifying *ServerWrites* step from *CR* to store *srvHist*, *srvAck*, *srvSent*, *srvMaxWriteSeqNumReceived* and on each server

$ModifiedCRServerWrites(op, s) \triangleq$
$\land\ op.id \in WID$
$\land\ op.id \notin srvReplied[s].w$
$\land\ srvReplied' = [srvReplied$ EXCEPT $![s] = [@$ EXCEPT $!.w = @ \cup \{op.id\}]]$
$\land\ srvHist' = [srvHist$ EXCEPT $![s] = Append(@, op)]$
$\land\ srvVals' = [srvVals$ EXCEPT $![s] = op.v]$
$\land\ \lor\ \land\ s + 1 > Current_Tail$
$\land\ resps' = Append(resps, [id \mapsto op.id])$
$\land\ srvAck' = [srvAck$ EXCEPT $![s] = @ \cup \{op.id\}]$
$\land\ srvReqs' = srvReqs$
$\land$ IF $False_Tail > Current_Tail$
THEN
Start adding updates to Sent in *Tail* when *syncing* with a new

```
                server
                srvSent' = [srvSent EXCEPT ![s] = Append(@, op)]
            ELSE
                srvSent' = srvSent
        ∧ UNCHANGED ⟨srvMaxWriteSeqNumReceived⟩
    ∨ ∧ s + 1 ≤ Current_Tail
        ∧ srvSent' = [srvSent EXCEPT ![s] = Append(@, op)]
        ∧ srvReqs' = [srvReqs EXCEPT ![s + 1] = Append(@, op)]
        ∧ srvMaxWriteSeqNumReceived' = [srvMaxWriteSeqNumReceived EXCEPT
![s + 1] = srvMaxWriteSeqNumReceived[s + 1] + 1]
            ∧ resps' = resps
            ∧ UNCHANGED ⟨srvAck⟩
    ∧ UNCHANGED ⟨reqs, NUM_SERVERS_ADDED, NUM_SERVERS_DELETED⟩

ModifiedCRServerReads(op, s) ≜
    ∧ op.id ∈ RID
    ∧ op.id ∉ srvReplied[s].r
    ∧ IF s = Current_Tail
        THEN
            ∧ srvReplied' = [srvReplied EXCEPT ![s] = [@ EXCEPT !.r = @ ∪
{op.id}]]
            ∧ resps' = Append(resps, [id ↦ op.id, v ↦ srvVals[s]])
            ∧ UNCHANGED ⟨srvVals, srvReqs, reqs, NUM_SERVERS_ADDED,
NUM_SERVERS_DELETED⟩
        ELSE
            ∧ srvReqs' =
                [
                    i ∈ 1 .. False_Tail
                    ↦
                    IF
                        i = Current_Tail
                    THEN
                        Append(srvReqs[i], op)  Sending read to Tail, if a
non − Tailhasreadsinitss rvReqs(fromearliertime, whenitwasaTAILitself)
                    ELSE
                        IF
                            i = s
                        THEN
                            Tail(srvReqs[i])  Removing read from current
srvReqs, asthisserverisnotaTailandcannotservicereads/queries
                    ELSE
                        srvReqs[i]

                ]
            ∧ UNCHANGED ⟨srvVals, reqs, resps, srvReplied, NUM_SERVERS_ADDED,
```

$NUM_SERVERS_DELETED\rangle$

After *Tail* finishes a write, it adds that write to its local *srvAck*. This ack is back propagated back through the chain

$BackPropagateAck(s) \triangleq$
$\land\ s > HEAD$
$\land\ Cardinality(srvAck[s] \setminus srvAck[s-1]) > 0$
$\land\ srvAck' = [srvAck \text{ EXCEPT } ![s-1] = srvAck[s]]$

For each server, head of *srvSent* is removed if an ack for that write is received and present in its *srvAck*

$UpdateSentFromAck(s) \triangleq$
$\land\ Len(srvSent[s]) > 0$
$\land\ Head(srvSent[s]) \in srvAck[s]$
$\land\ srvSent' = [srvSent \text{ EXCEPT } ![s] = Tail(@)]$

Steps related to read and write ops

$ModifiedCRNext \triangleq$
$\lor \exists\, rid \in RID :$
$\land\ ModifiedCRIssueRead(rid)$
$\land\ STEP_TAKEN' =$ "ModifiedCRIssueRead"
$\land\ REMOVED_INDICES' = \{\}$
$\land$ UNCHANGED $\langle srvAck, srvHist, srvSent, srvMaxWriteSeqNumReceived, Current_Tail, False_Tail\rangle$
$\lor \exists\, wid \in WID :$
$\land\ ModifiedCRIssueWrite(wid)$
$\land\ STEP_TAKEN' =$ "IssueWrite"
$\land\ REMOVED_INDICES' = \{\}$
$\land$ UNCHANGED $\langle srvAck, srvHist, srvSent, Current_Tail, False_Tail\rangle$
$\lor \exists\, s \in (1 \,..\, Current_Tail) :$
$\land\ Len(srvReqs[s]) > 0$
$\land$ LET $o \triangleq Head(srvReqs[s])$ IN

$\lor$
$\land\ DropRead(o, s) \land STEP_TAKEN' =$ "DropRead" $\land$
$REMOVED_INDICES' = \{\} \land$ UNCHANGED $\langle srvAck, srvHist, srvSent, srvMaxWriteSeqNumReceived, Current_Tail, False_Tail\rangle$
$\lor$
$\land\ DropWrite(o, s) \land STEP_TAKEN' =$ "DropWrite" $\land$
$REMOVED_INDICES' = \{\} \land$ UNCHANGED $\langle srvAck, srvHist, srvSent, srvMaxWriteSeqNumReceived, Current_Tail, False_Tail\rangle$

$\lor$
$\land\ ModifiedCRServerReads(o, s)$
$\land\ STEP_TAKEN' =$ "ModifiedCRServerReads"
$\land\ REMOVED_INDICES' = \{\}$
$\land$ UNCHANGED $\langle srvAck, srvHist, srvSent,$

$srvMaxWriteSeqNumReceived$, $Current_Tail$, $False_Tail\rangle$

$\lor$

$\land$ $ModifiedCRServerWrites(o, s)$

$\land$ $STEP_TAKEN' =$ "ModifiedCRServerWrites"

$\land$ $REMOVED_INDICES' = \{\}$

$\land$ UNCHANGED $\langle Current_Tail, False_Tail\rangle$

$\lor$

$\land \exists\, s \in (1 \,..\, Current_Tail) : BackPropagateAck(s)$

$\land$ $STEP_TAKEN' =$ "BackPropagateAck"

$\land$ $REMOVED_INDICES' = \{\}$

$\land$ UNCHANGED $\langle reqs, resps, srvReplied, srvReqs, srvVals, srvHist, srvSent, srvMaxWriteSeqNumReceived, Current_Tail, False_Tail, NUM_SERVERS_ADDED, NUM_SERVERS_DELETED\rangle$

$\lor$

$\land \exists\, s \in (1 \,..\, Current_Tail) : UpdateSentFromAck(s)$

$\land$ $STEP_TAKEN' =$ "UpdateSentFromAck"

$\land$ $REMOVED_INDICES' = \{\}$

$\land$ UNCHANGED $\langle reqs, resps, srvReplied, srvReqs, srvVals, srvAck, srvHist, srvMaxWriteSeqNumReceived, Current_Tail, False_Tail, NUM_SERVERS_ADDED, NUM_SERVERS_DELETED\rangle$

Next steps of the spec where it can cause failures of maximum $N-1$ servers in a N length chain or continue normally with read/write operations. With each step, a $STEPS$ counter is always increased

$AddFailStopCRNext \triangleq$

$\land$ $STEPS' = STEPS + 1$

$\land$

$\lor$ $FailureAndRecoveryStep$

$\lor$ $ModifiedCRNext$

$\lor \exists\, num_servers \in \{1, 2\} :$ $AddNewServertoChain(num_servers)$

$\lor$ $SyncHistoryWithNewServer$

$\lor$ $ShiftCurrentTailToNewServer$

$vars \triangleq \langle srvReplied, srvVals, srvReqs, resps, reqs, STEPS, Current_Tail, REMOVED_INDICES, srvHist, srvSent, srvAck, srvHist, STEP_TAKEN, srvMaxWriteSeqNumReceived, False_Tail, NUM_SERVERS_ADDED, NUM_SERVERS_DELETED\rangle$

$AddFailStopFairness \triangleq \mathrm{WF}_{vars}(AddFailStopCRNext)$

$AddFailStopLiveness \triangleq$

$\land \forall\, req \in reqs :$ $\Box\Diamond(req \in SequencetoSet(resps))$

$EventuallySyncwithNewServers \triangleq$
$\quad \Box\Diamond(Current_Tail = False_Tail)$

Final Spec of Fail Stop CR

$AddFailStopCRSpec \triangleq$
$\quad \wedge\ AddFailStopCRInit$
$\quad \wedge\ \Box[AddFailStopCRNext]_{vars}$
$\quad \wedge\ AddFailStopFairness$
$\quad \wedge\ AddFailStopLiveness$
$\quad \wedge\ EventuallySyncwithNewServers$